

Pipelining Fundamentals & the 5-Stage RISC Pipeline

AArch64 instruction flow, pipeline state,
hazards, and performance

Dr. J. Burns

PART 1

Pipelining Principles and Timing

The AArch64 Teaching Model

- **ISA behavior:** execute real A64 instructions while preserving their register, memory, and control-flow effects.
- **Implementation:** a single-issue, in-order pipeline with stages IF, ID, EX, MEM, and WB.
- **Timing assumptions:** one cycle per stage, separate instruction/data access, cache hits, and branch resolution in EX.
- **Register timing:** a WB write is visible to an ID read in the same cycle.
- **Scope:** simple integer ALU operations, offset `ldr/str`, and `b/cbz/cbnz`. Use `x0–x30` in the worked examples.

ISA and Microarchitecture

The AArch64 ISA does not prescribe five stages. The diagrams describe the teaching implementation used in this lecture.

Overlapping Instructions

Single-cycle: one complete instruction per long cycle

I1: all five stages

I2: all five stages

I3: all five stages

Pipelined: different instructions use different stages

I5: IF

I4: ID

I3: EX

I2: MEM

I1: WB

One snapshot of a full pipeline. Each instruction advances at the next clock edge.

- The hardware overlaps work on several instructions without changing program order.
- The shorter clock permits more instructions to finish per unit of time once the pipeline is full.

Clock Period and Stage Balance

Stage	IF	ID	EX	MEM	WB
Delay (picoseconds, ps)	200	100	200	200	100

One picosecond is 10^{-12} seconds. Assume 20 ps of register and clocking overhead.

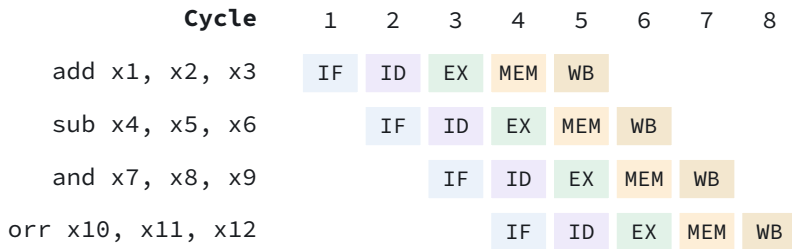
Illustrative Timing Calculation

Single-cycle period: $T_{\text{single}} = \sum t_{\text{stage}} + t_{\text{reg}} = 800 + 20 = 820$ ps.

Pipeline period: $T_{\text{pipe}} = \max(t_{\text{stage}}) + t_{\text{reg}} = 200 + 20 = 220$ ps.

- The slowest stage limits the clock. Spare time in ID and WB does not shorten a 200 ps stage.
- For a long independent stream, the ideal speedup approaches $820/220 \approx 3.73$.

Reading a Pipeline Timing Diagram



- Read a row to follow one instruction. Read a column to see the work occurring in one cycle.
- Four independent instructions take $4 + 4 = 8$ cycles from first fetch to final WB.
- The initial fill and final drain prevent a short sequence from achieving one instruction per cycle overall.

Latency, Throughput, and Ideal CPI

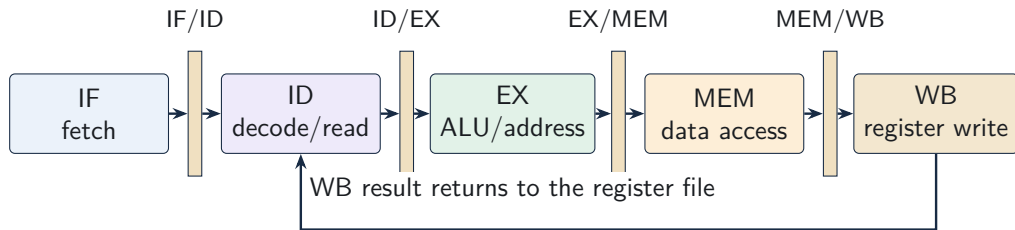
Quantity	Meaning in this model
Instruction latency	Five cycles from IF through WB: $5 \times 220 = 1100$ ps.
Steady-state throughput	One instruction reaches WB every 220 ps when there are no hazards.
Finite sequence	N instructions require $N + 4$ cycles in an initially empty pipeline.
Cycles per instruction (CPI)	$\text{CPI} = (N + 4)/N$ without stalls or flushes.

Pipelining Improves Throughput

The example pipeline has a longer per-instruction latency than the 820 ps single-cycle design, but finishes a long instruction stream sooner.

The Five Stages and Pipeline State

The 5-Stage Datapath



- Pipeline registers hold each stage's outputs until the next clock edge.
- Different instructions occupy these stages concurrently.
- An instruction passes through every stage, but only enables the actions it needs.

IF and ID: Fetch, Decode, and Read

IF: Instruction Fetch

- Use the PC to fetch one 32-bit A64 instruction.
- Select $PC + 4$ for the next sequential instruction.
- Keep the instruction's own PC in IF/ID for later PC-relative calculations.

ID: Decode and Register Read

- Identify the instruction's source and destination registers.
- Read operands and generate an immediate value.
- Produce control signals and record them in ID/EX.

`ldr x7, [x5, #16]`: read x5, record destination x7, and generate the byte offset 16.

EX: Arithmetic, Addresses, and Branches

Instruction	EX-stage operation
add x7, x5, x6	Add the two operand values; result is destined for x7.
ldr x7, [x5, #16]	Compute effective address $x5 + 16$.
str x7, [x5, #16]	Compute effective address $x5 + 16$; preserve store data separately.
cbz x7, .Lzero	Test the operand for zero and calculate the target from the branch's PC.

Values Needed in EX

The forwarding network can replace an old ID operand with a newer result. Store data and the cbz/cbnz test operand also use this network in our model.

MEM and WB: Memory Effects and Register Results

MEM: Data Memory Access

- `ldr`: read memory at the effective address.
- `str`: write the saved store-data value to memory.
- ALU instructions and branches pass through without data-memory access.

WB: Register Writeback

- Select the ALU result or loaded memory value.
- Write the saved destination register if `RegWrite = 1`.
- `str` and the branches used here have `RegWrite = 0`.

An ALU result is available at the end of EX. A loaded value becomes available at the end of MEM.

Pipeline Registers Carry Data and Control

Boundary	Information retained for the same instruction
IF/ID	Instruction word, instruction PC, valid bit.
ID/EX	Operand values, source/destination register numbers, immediate, PC, EX/MEM/WB controls.
EX/MEM	ALU result or address, corrected store data, destination number, MEM/WB controls.
MEM/WB	Loaded data, ALU result, destination number, WB controls.

- For `ldr x7, [x5, #16]`, destination number 7 must travel all the way to WB.
- WB cannot use the destination field of the newer instruction currently in ID.
- A valid bit or disabled write controls allows an empty slot to move safely through the pipeline.

Worked Load and Store: The Same Address Path

Assume: $x5 = 0x1000$, memory at $0x1010$ holds 42, and $x7 = 99$ initially.

Stage	ldr x7, [x5, #16]	str x7, [x5, #16]
IF	Fetch the instruction.	Fetch the instruction.
ID	Read x5; save destination 7.	Read x5 and store source x7.
EX	Address = $0x1010$.	Address = $0x1010$; store data = 99.
MEM	Read the 64-bit value 42.	Write the 64-bit value 99.
WB	Write 42 into x7.	No register write.

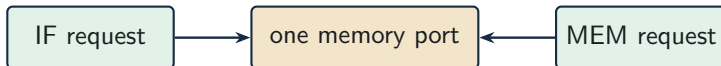
Treat the two examples independently. The unsigned-offset encoding has $\text{imm12} = 2$, giving $2 \times 8 = 16$ bytes; x5 is unchanged.

PART 3

Hazards, Forwarding, and Stalls

Three Classes of Pipeline Hazard

Hazard	What prevents normal progress?	Example
Structural	Two stages need the same hardware resource at once.	IF and MEM compete for one memory port.
Data	A consumer needs a result that an older instruction has not yet supplied.	add produces the operand of a following sub.
Control	The correct next PC is not yet known.	cbz may redirect instruction fetch.



Our model avoids this structural conflict with separate instruction and data access.

RAW Dependence: Waiting for a Register Result

A read-after-write (RAW) dependence

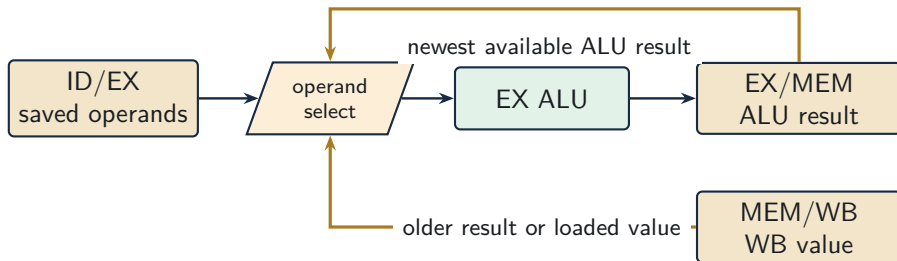
```
add x7, x5, x6
sub x8, x7, x9
```



- Without forwarding, sub must wait for add to write x7.
- WB writes before ID reads in cycle 5, so the consumer reaches EX in cycle 6: two stall cycles.
- ID* marks an instruction held in ID. The dependency is on a value, not just adjacent instructions.

Forwarding an ALU Result

add x7, x5, x6 followed by sub x8, x7, x9

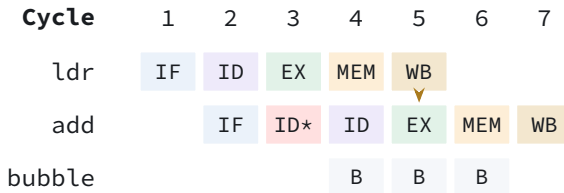


- add finishes EX in cycle 3. EX/MEM supplies its result to sub in EX in cycle 4: no stall.
- Match valid register writes to source operands. If two results match, select the newest one.
- Never forward a load's address as loaded data.

The Load-Use Hazard: One Cycle Too Soon

An immediate consumer of a loaded value

```
ldr x7, [x5]
add x8, x7, x9
```



- The load produces data at the end of cycle 4. The consumer cannot use it at the start of EX in that same cycle.
- Hold the consumer for one cycle, then forward from MEM/WB to EX in cycle 5.
- B = an inactive slot, with no register or memory write.

Stall Control and Instruction Scheduling

Hardware Response to Load-Use

- 1 Hold the PC and IF/ID register.
- 2 Insert an invalid slot into ID/EX.
- 3 Let the older load advance normally.

Independent work fills the gap

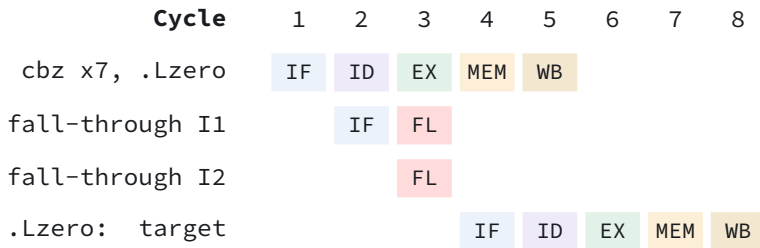
```
ldr x7, [x5]
add x10, x11, x12
add x8, x7, x9
```

- In the reordered sequence, the load reaches MEM in cycle 4 and the dependent add reaches EX in cycle 5. Forwarding supplies the value without a stall.
- Reordering must preserve all register and memory dependencies and other observable effects.
- Our model obtains store data in EX, with no special late MEM bypass. A load immediately followed by a store of that loaded value therefore also needs one stall.

Control Flow and Pipeline Performance

A Taken Branch Flushes Younger Instructions

Assume: predict sequential fetch; resolve cbz in EX; the tested value is zero.



- At the end of cycle 3, redirect the PC and invalidate the younger instruction in ID and the instruction fetched in IF.
- Target fetch begins in cycle 4. Two instruction slots are lost under these assumptions.
- FL marks discarded work. Flushes must prevent wrong-path register and memory writes.

AArch64 Branch Operands and Condition Flags

Test a general-purpose register

```
cbz x7, .Lzero  
cbnz x7, .Lnonzero
```

- Test the value of x7 directly.
- Do not read or update NZCV.
- The test operand may need forwarding or a load-use stall.

Test condition flags

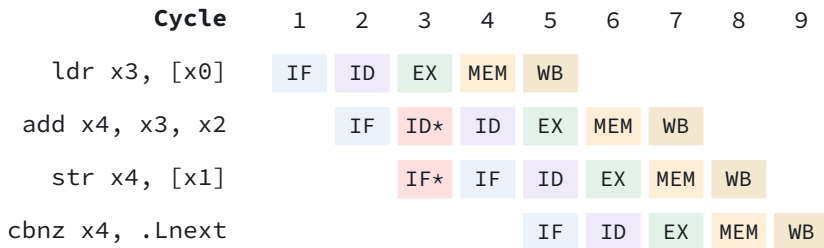
```
cmp x7, x8  
b.eq .Lequal
```

- cmp updates NZCV; b.eq tests Z.
- cmp is an alias of subs with a zero-register destination.
- Extending the datapath for this pair requires flag dependence handling.

Use the branch instruction's saved PC for the target. For cbz/cbnz, the signed encoded displacement is multiplied by 4.

Worked Sequence: Load, Add, Store, and Branch

Assume: the branch is not taken. Forwarding includes store data and the branch test operand.



- ldr to add: one stall, then MEM/WB-to-EX forwarding.
- add to str: forward x4 from EX/MEM into the store-data path in cycle 6.
- add to cbnz: forward x4 from MEM/WB into the branch test in cycle 7.
- The pipeline drains in $4 + 4 + 1 = 9$ cycles. Store and branch WB slots perform no register write.

Counting Lost Cycles and Execution Time

Finite Run Under the Stated Model

$$\text{cycles} = N + 4 + S + 2B$$

S : load-use stall cycles. B : taken branches predicted not taken.

Assume the counted penalties do not overlap and occur before the final drain.

Illustrative run	Calculation
100 instructions; 10 load-use stalls; 10 taken branches	$100 + 4 + 10 + 2(10) = 134$ cycles.
Average CPI	$134/100 = 1.34$.
Pipelined time (nanoseconds, ns)	$134 \times 220 \text{ ps} = 29.48 \text{ ns}$.
Single-cycle baseline and speedup	82 ns; speedup $82/29.48 \approx 2.78$.

Hazards reduce the achieved speedup below the ideal long-stream value of 3.73.

What the Simple Model Leaves Out

- **Longer operations:** cache misses and multi-cycle arithmetic can hold a stage for several cycles.
- **Branch prediction:** real implementations may predict both direction and target. Incorrect predictions discard work.
- **Register ordering:** this model reads in ID and writes in WB, in program order. Out-of-order designs need additional bookkeeping.
- **AArch64 register views:** w7 and x7 refer to the same register; a write to w7 clears the upper 32 bits of x7.
- **More complex instructions:** pre/post-indexed loads, paired transfers, and flag-setting operations need additional datapath and control support.

Topic 9 Summary

- **Pipelining:** overlap instructions to increase throughput. The five stages are IF, ID, EX, MEM, and WB.
- **Clock and timing:** the slowest stage plus register overhead sets the clock period. An ideal N -instruction run takes $N + 4$ cycles.
- **Pipeline state:** retain each instruction's operands, PC, destination, and control signals across stage boundaries.
- **Data hazards:** forward available results. In our model, an immediate load-use dependence still needs one stall.
- **Control hazards:** an EX-resolved taken branch flushes two younger slots under sequential prediction.
- **Correctness:** a store writes memory rather than a register. Bubbles and flushed instructions enable no architectural writes.

Self-Test 1: Principles and Timing

- 1 Explain the difference between instruction latency and instruction throughput. Which does pipelining primarily improve?
- 2 Stage logic delays are 200, 100, 200, 200, and 100 ps; pipeline-register overhead is 20 ps. Calculate the pipeline clock period and five-stage instruction latency.
- 3 Eight independent instructions execute in an initially empty five-stage pipeline. How many cycles are needed through final WB? What is the average CPI?
- 4 Draw a cycle diagram for three independent AArch64 ALU instructions. Which stages are occupied in cycle 3?
- 5 Why is a five-stage pipeline a microarchitectural choice rather than a requirement of the AArch64 ISA?

Self-Test 2: Datapath and Data Hazards

- 1 Trace `ldr x7, [x5, #16]` through all five stages. If `x5 = 0x1000`, what address is read? What is the unsigned-offset `imm12` field?
- 2 For `str x7, [x5, #16]`, identify the base register and store-data register. Which stage writes memory, and why is `RegWrite = 0`?
- 3 `add x7, x5, x6` supplies `x7` to a following `sub`. Give the forwarding path and stall count. Repeat with `ldr x7, [x5]` as the producer; both consumers need `x7` in EX.
- 4 During a load-use stall, which two state elements are held, where is the bubble inserted, and why must the older load continue?
- 5 Why must a hazard detector recognize a dependence from a write to `w7` to a later read of `x7`?

Self-Test 3: Control Flow and Performance

- 1 A taken `cbz` resolves in EX in cycle 3 under sequential prediction. Which younger slots are flushed, and in which cycle can target fetch begin?
- 2 How do the operands of `cbz x7, .Lzero` differ from those of `cmp x7, x8; b.eq .Lequal`? Which sequence depends on NZCV?
- 3 For `ldr x3, [x0]; add x4, x3, x2; str x4, [x1]`, draw the timing diagram with forwarding. Identify the stall and the path supplying store data.
- 4 A 100-instruction run has 8 load-use stalls and 6 taken branches predicted not taken. Assume isolated penalties and full drain. Calculate cycles, CPI, and execution time with a 220 ps clock.
- 5 Why must flushing a wrong-path `str` disable its memory write? Would clearing only its register-write control be sufficient?